Compte-rendu de travaux pratiques

Exemple de feux rouges

LED pilotées par une carte STM32F407VGTx

Réalisé par	Mohamed Amine Dhaoui Ahmed Hafi Ayetallah Cherif Essra Gammoudi
Domaine	Systèmes embarqués / microcontrôleurs
Plateforme	STM32F407VGTx (ARM Cortex-M4, 168 MHz)
Environnement	STM32CubeIDE, programmation en langage C par accès direct aux registres (CMSIS)

Note de lecture. Le code présenté est celui réellement exécuté sur la carte ; le câblage et les résultats proviennent des photographies et de la vidéo du montage. Les durées réelles des phases dépendent d'une boucle d'attente non calibrée (voir étape 4).

1. Contexte et objectif

Ce TP a pour but de piloter trois LED externes (rouge, jaune, verte), implantées sur une plaque d'essai, à partir des sorties GPIO d'un microcontrôleur STM32F407VGTx, afin de reproduire le cycle d'un feu de circulation. Il met en œuvre la configuration de broches en sortie, la temporisation logicielle, ainsi que le câblage et le dimensionnement d'un circuit de protection de LED.

Matériel utilisé

Élément	Détail
Carte de développement	STM32F407VGTx, alimentée et programmée par USB via le ST-LINK/V2 intégré
Plaque d'essai	Breadboard 830 points avec rails d'alimentation (+ / −)
LED	1 verte, 1 orange (jaune), 1 rouge (5 mm)
Résistances	3 × environ 220 Ω (bagues rouge-rouge-marron)
Connectique	Câbles Dupont (rouge, blanc, violet), câble USB

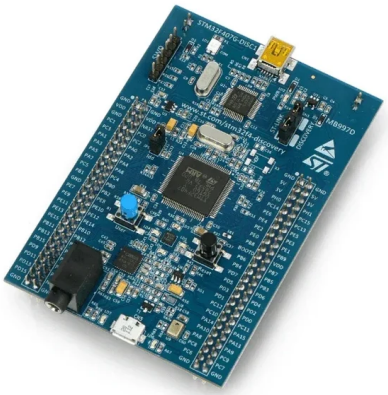


Figure 1 : Carte de développement STM32F407VGTx utilisée dans ce TP.

2. Déroulement étape par étape

Étape 1 : Dimensionnement du circuit de protection

Une LED ne doit jamais être connectée directement à une sortie GPIO : sans limitation, le courant dépasserait les capacités de la broche. Chaque LED est donc placée en série avec une résistance. Pour une sortie de 3,3 V, une tension directe typique $V_F \approx 2$ V et R = 220 Ω :

$$\mathbf{I = (V_{DD} - V_F) / R = (3{,}3 - 2{,}0) / 220 \approx 5{,}9\ mA}$$

Ce courant est très inférieur à la limite d'une broche du STM32F4 (25 mA) et suffit à obtenir une luminosité correcte.

Étape 2 : Câblage sur la plaque d'essai

- Chaque **anode** de LED est reliée, via sa résistance de 220 Ω, à une sortie GPIO de la carte par un câble Dupont.
- Les **cathodes** des trois LED sont ramenées au rail de masse (−), relié à une broche GND de la carte.
- Les trois résistances sont implantées de part et d'autre de la gouttière centrale.

LED	Broche STM32	Masque dans le code
Verte	PD12	LED_VERT = 1U << 12
Orange	PD13	LED_ORANGE = 1U << 13
Rouge	PD14	LED_ROUGE = 1U << 14

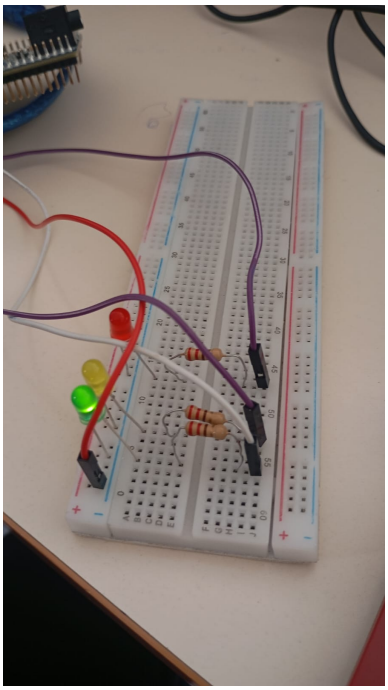


Figure 2 : Montage hors tension. LED rouge, jaune et verte ; résistances ; fils vers la carte.

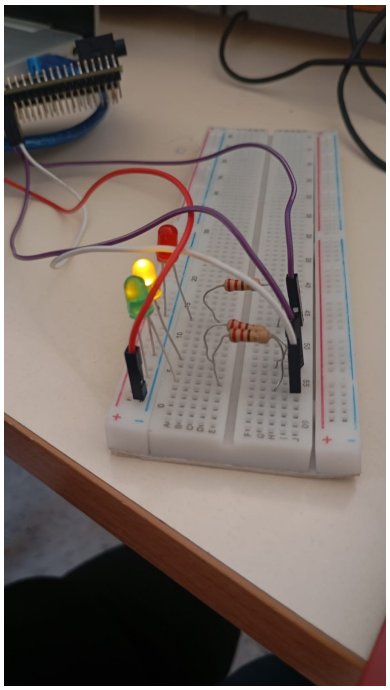


Figure 3 : Montage sous tension. LED jaune vive et LED verte plus faible.

Étape 3 : Configuration de l'environnement logiciel

- Création d'un projet STM32CubeIDE pour la cible STM32F407VGTx, avec l'en-tête CMSIS `stm32f4xx.h`.
- Activation de l'horloge du port GPIOD via `RCC->AHB1ENR` : sans elle, le port ne fonctionne pas.
- Configuration de PD12, PD13 et PD14 en **sortie** (`MODER` = 01) de type **push-pull** (`OTYPER` = 0).

Étape 4 : Écriture du code

Le programme manipule directement les registres du port GPIOD, sans bibliothèque HAL. Les trois LED sont repérées par des masques binaires (bits 12 à 14). La fonction `allumer()` utilise le registre BSRR : les bits 16 à 31 éteignent les trois LED, puis les bits 0 à 15 allument celle demandée, ce qui garantit qu'une seule LED est active à la fois. Le cycle est : vert, orange, rouge, puis retour au vert.

```c
#include "stm32f4xx.h"            // Inclut les définitions des registres du STM32F4 (GPIOD, RCC, etc.)

#define LED_VERT     (1U << 12)   // LED verte sur PD12 : bit 12 mis à 1 (masque binaire)
#define LED_ORANGE   (1U << 13)   // LED orange sur PD13 : bit 13 mis à 1
#define LED_ROUGE    (1U << 14)   // LED rouge sur PD14 : bit 14 mis à 1
#define TOUTES_LED   (LED_VERT | LED_ORANGE | LED_ROUGE)  // Masque regroupant les 3 LED (bits 12, 13, 14)

/* Temporisation approximative (boucle d'attente) */
static void delay(volatile uint32_t time)  // Fonction locale (static) qui attend "time" tours de boucle
{                                 // Début de la fonction delay
    while (time--)                // Tant que time n'est pas 0, on le décrémente et on boucle
    {                             // Début du corps de la boucle
    }                             // Corps vide : la boucle ne fait que perdre du temps
}                                 // Fin de la fonction delay

/* Allume uniquement la LED demandée, éteint les autres */
static void allumer(uint32_t led) // Fonction locale qui allume une seule LED passée en paramètre
{                                 // Début de la fonction allumer
    GPIOD->BSRR = (TOUTES_LED << 16);
        // Écrit dans les bits 16 à 31 de BSRR : remet à 0 (éteint) PD12, PD13, PD14
    GPIOD->BSRR = led;            // Écrit dans les bits 0 à 15 de BSRR : met à 1 (allume) la LED choisie
}                                 // Fin de la fonction allumer

int main(void)                    // Point d'entrée du programme
{                                 // Début de main
    /* 1. Activer l'horloge du port D */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;
        // Active l'horloge du port GPIOD (sans elle, le port ne fonctionne pas)

    /* 2. PD12, PD13, PD14 en sortie (MODER = 01) */
    GPIOD->MODER &= ~((3U << (12 * 2)) | (3U << (13 * 2)) | (3U << (14 * 2)));
        // Met à 00 les 2 bits de mode de PD12, PD13, PD14 (remise à zéro)
    GPIOD->MODER |=  ((1U << (12 * 2)) | (1U << (13 * 2)) | (1U << (14 * 2)));
        // Met ces bits à 01 : broches configurées en sortie

    /* 3. Sortie push-pull */
    GPIOD->OTYPER &= ~TOUTES_LED; // Met les bits 12, 13, 14 à 0 : type de sortie push-pull (et non open-drain)

    /* 4. Cycle des feux */
    while (1)                     // Boucle infinie : le cycle des feux se répète sans fin
    {                             // Début de la boucle principale
        allumer(LED_VERT);        // Allume la LED verte, éteint les autres
        delay(3000000);           // Attend longtemps : feu vert affiché

        allumer(LED_ORANGE);      // Allume la LED orange, éteint les autres
        delay(1000000);           // Attend moins longtemps : feu orange affiché

        allumer(LED_ROUGE);       // Allume la LED rouge, éteint les autres
        delay(3000000);           // Attend longtemps : feu rouge affiché
    }                             // Fin de la boucle infinie (on retourne au début)
}                                 // Fin de main (jamais atteinte)
```

Remarques : la fonction `delay()` est une temporisation logicielle approximative : sa durée dépend de la fréquence d'horloge et de l'optimisation du compilateur (ici 3 000 000 tours pour le vert et le rouge, 1 000 000 pour l'orange). Par ailleurs, PD12, PD13 et PD14 pilotent aussi les LED intégrées à la carte (LD4, LD3, LD5), qui s'allument donc en même temps que les LED externes.

Étape 5 : Compilation, flashage et test

- Compilation du projet (*Build*) sans erreur.
- Connexion de la carte au PC par USB (câble bleu visible sur les photos) et flashage via le ST-LINK.
- Exécution du programme et observation du comportement des LED.

3. Résultats

Le montage est fonctionnel : les LED s'allument de façon séquentielle sous le contrôle du microcontrôleur, avec une luminosité satisfaisante grâce aux résistances de 220 Ω. La vidéo de démonstration (durée : environ 12 s) est restituée par six images extraites à intervalles réguliers (figures 4 à 9).

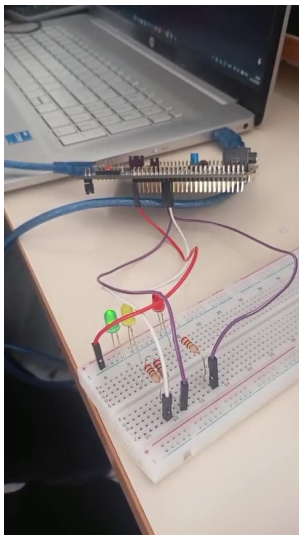


Figure 4 : t ≈ 1 s, LED orange allumée.

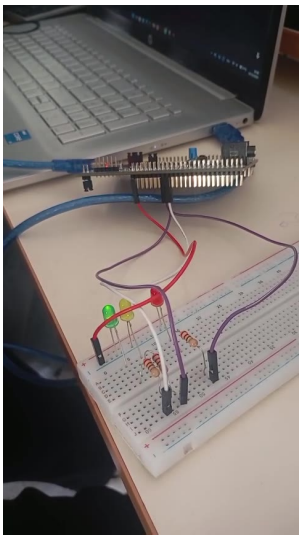


Figure 5 : t ≈ 4 s, LED orange allumée.

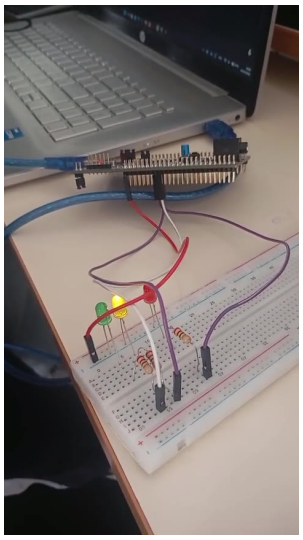


Figure 6 : t ≈ 6 s, LED orange allumée.

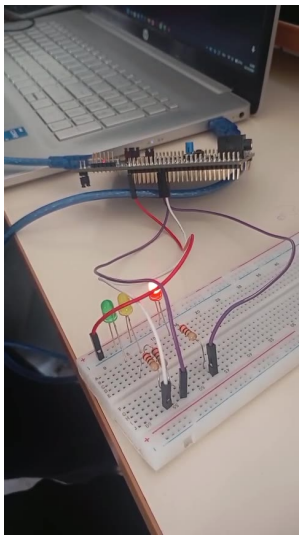


Figure 7 : t ≈ 8 s, passage à la LED rouge.

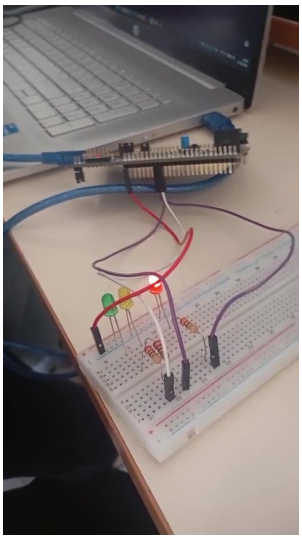


Figure 8 : t ≈ 10 s, LED rouge allumée.

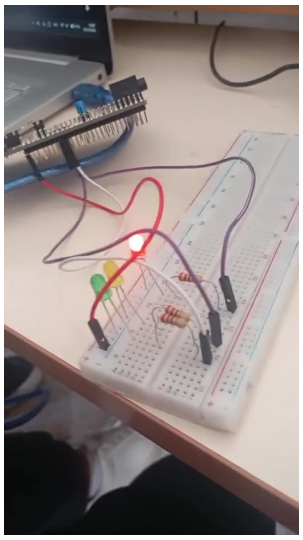


Figure 9 : t ≈ 11,5 s, vue rapprochée, LED rouge.

Analyse. La vidéo montre le passage d'une phase dominée par la LED jaune à une phase où la LED rouge est allumée, ce qui est cohérent avec l'enchaînement vert, orange, rouge programmé. Sur certaines images, la LED verte apparaît faiblement éclairée en même temps que l'orange (voir aussi la figure 3) : il s'agit vraisemblablement de la transition entre deux phases ou d'un effet de persistance de la caméra, car le code n'allume qu'une LED à la fois. Les durées exactes des phases n'ont pas été mesurées.

4. Conclusion et bilan

Objectifs validés

- Configuration correcte de PD12, PD13 et PD14 en sortie push-pull par accès direct aux registres.
- Câblage fonctionnel, LED protégées par des résistances de 220 Ω (environ 6 mA par LED).
- Commande séquentielle des LED conforme au fonctionnement d'un feu de circulation.

Points clés retenus

- Une résistance de limitation est indispensable entre une LED et une sortie GPIO.
- La masse doit être commune entre le circuit externe et la carte.
- L'horloge du port doit être activée avant toute configuration ; le registre BSRR permet de modifier les broches de façon atomique.

Difficultés fréquentes et solutions

Difficulté	Solution
LED qui ne s'allume pas	Vérifier la polarité (patte longue = anode) et la continuité de la masse
LED allumée faiblement ou en permanence	Contrôler le mode de la broche et l'absence de conflit avec une fonction alternative
Câble Dupont mal enfiché	Tester la continuité et réenfoncer les connecteurs

Perspectives d'amélioration

- Remplacer la boucle `delay()` par un timer matériel (SysTick ou TIMx) pour des durées précises et non bloquantes.
- Ajouter un bouton piéton sur le bouton utilisateur (PA0).
- Implémenter une machine à états pour étendre le système à un carrefour à deux voies.